

PAGES facilities.html -> facilities.php

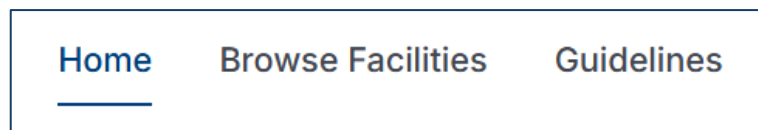
For this page, only 1. & 2. is important to let you know the logic, the rest you probably dah tau and can do sendiri. I include je, but don't necessarily need to follow. There might be comments on the top part of the HTML pages, delete je nanti. This doc is for us to keep track of front/backend. Senang for you to refer. Kalau you dah siap the parts, just highlight them all. Feel free to edit this doc.

1. Dynamic Navigation Bar (Session Check)

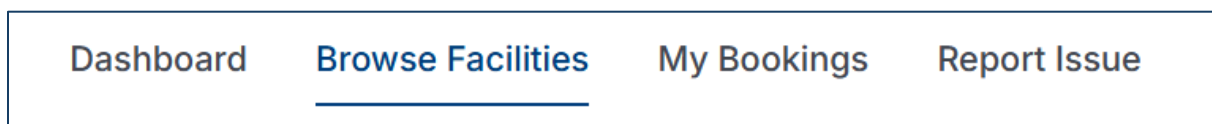
Because this page is accessible from both the public Landing Page and the Student Dashboard, the Navbar needs to change based on the user's session.

- **Action:** Write an if/else statement checking the PHP Session.
- **IF logged in:** (isset(\$_SESSION['user_id'])) -> include the Student/Logged-in Navbar (shows Dashboard, My Bookings, User Profile).
- **ELSE:** -> include the Public Navbar (shows Home, Sign In, Activate Account).

Public Navigation Bar from index.html can Browse Facilities:



Private Navigation Bar from student-dashboard to Browse Facilities:



2. Keep in mind, dekat index.html page, theres a design that says:

2	5+	40+	0
CAMPUSES	FACILITY TYPES	BOOKABLE SPACES	PHYSICAL QR CODES

So nanti bila buat database, kalau there isn't 5+ facility types...then change the number. Instead of 5+ maybe jadi 3+ only. Same goes to the 40+ bookable spaces.

I think ni boleh sub kat Suriya. You complete logic of database, while Suriya cari data untuk letak. Like the names, locations of facilities, images to use and so on. Dia deal ngan SQL je.

3. Dynamic Facility Rendering (Database Loop)

The HTML currently has 3 hardcoded facility cards as placeholders.

- **Action:** Delete the hardcoded cards and replace them with a PHP while loop that fetches data from the FACILITY table.
- **Data Mapping:** Map the database columns to the HTML card:
 - facility_name -> Goes into the <h3> tag.
 - location -> Goes into the Campus location .
 - capacity -> Goes into the Max Capacity .
 - category -> Can be used for filtering.

4. Search & Filter Bar Logic

The filter bar at the top has method="GET".

- **Action:** When the user clicks "Filter", grab the \$_GET['campus'], \$_GET['category'], and \$_GET['search'] values.
- Modify your SQL query to include WHERE and LIKE clauses to dynamically filter the facilities displayed on the page.

5. "Book Now" Button Logic & Redirection

The "Book Now" button on each card needs to know *which* facility was clicked and *who* is clicking it.

- **Action:** Change the button link to pass the specific Facility ID via URL parameter (e.g., href="booking-form.php?id=php echo \$row['facility_id']; ?>").</li- **Auth Check:** If a *public/unauthenticated* user clicks "Book Now", intercept the click, redirect them to login.php, and show a session error message: *"Please log in to book this facility."*

PAGES booking-form.html & booking-form-lecturer.html

1. General Quota & Time Rules (Applies to ALL users)

To ensure fair access across the campus, the system enforces strict quota and time limits.

- **Refresh Rate:** Quotas reset **weekly** (e.g., every Sunday at midnight maybe via a timestamp check). *Don't use per-semester refresh, not convenient for UX (too long).*
- **Amount: 2 Quotas per week** per user.
- **Cost:** 1 Booking = 1 Quota deducted, regardless of how long the booking is.
- **Duration Limit:** A booking must be a minimum of 1 hour and a maximum of 2 hours.
 - *Backend/JS Action:* When the user selects "End Time", calculate (End Time - Start Time). If the result is > 2 hours, block the submission and throw an error: "Maximum booking duration is 2 hours."

2. Lecturer "Priority Booking" Logic (Lecturer Form Only)

A "Priority Booking" is an emergency override used when a lecturer *must* have a room for official academic reasons (e.g., a replacement class or exam), even if a student has already booked it.

- **Standard Quota Use:** If a lecturer does *not* check the Priority box (e.g., they are booking a badminton court for personal use), they consume their normal 2-per-week quota, and the booking is **Auto-Approved**.
- **Priority Trigger:** If the lecturer *checks* the Priority box, the logic changes:
 - **No Quota Cost:** It does not consume their weekly quota.
 - **Not Automatic:** The booking status is forced to **Pending** and requires manual Admin approval.
 - **Mandatory Proof:** The file upload field and the "Purpose & Subject Code" text area must become strictly **Required (required attribute)**.
 - **Admin Override:** If the Admin approves it, the system must check if a student had that room booked. If yes, cancel the student's booking and send an automated email notification.
 - **Abuse Penalty:** If the Admin finds a lecturer abusing this feature (e.g., priority booking a sports court), the Admin will reject it and issue a Penalty Strike.

3. File Upload Handling & Frontend JS

To prevent abuse of the Priority system, lecturers must upload proof (Faculty Memo, Timetable Change, Approval Letter). This gives the Admin hard evidence before clicking "Approve".

- **Frontend UI:** Has Drag & Drop zone.
- **JavaScript:** Shows the file name when dropped/selected. JS is handled in a separate file: public/js/script.js to keep the HTML clean.
- *Backend Action:* When the form is submitted, PHP needs to handle the \$_FILES['proofUpload'] array, move the uploaded file to a secure server folder (e.g., public/uploads/proofs/), and save the file path in the BOOKING table in the database. Ensure file type validation (PDF, JPG, PNG) and a 5MB size limit are enforced on the PHP side for security.

4. "Add on equipments"

Should be different depending on the facility. Should match the facility displayed.

Eg. For lecture hall, takde equipment badminton net.